

MULEGUARD AI: LEAKAGE-HARDENED MONEY-MULE DETECTION AND A DATASET INTEGRITY AUDIT

1st Given Name Surname, 2nd Given Name Surname, 3rd Given Name Surname

Department Name
Institution Name
City, Country

ABSTRACT

Money-mule accounts settle digital payment fraud but are vanishingly rare in labelled banking data. We present MuleGuard AI, a detection pipeline for 9,082 accounts containing 81 confirmed mules (0.8919%). Our primary finding is that the benchmark itself is contaminated: positives and negatives are drawn from disjoint monthly extracts, and a model given only the pattern of blank cells—every value discarded—reaches AUPRC 0.824 against a 0.0089 random baseline. We supply falsification tests that make this measurable. We further contribute a leakage taxonomy that classifies fields by meaning rather than correlation, catching an outcome flag correlated just 0.05 with the target, and 29 named mule-typology features. Under nested repeated cross-validation, in which selection, calibration and the operating threshold are all fitted in-fold, the ensemble reaches precision 0.980 ± 0.043 at recall 0.719 ± 0.121 . We argue these are an upper bound on what this dataset can establish.

Index Terms— Money-mule detection, anti-money laundering, class imbalance, target leakage, dataset integrity

1. INTRODUCTION

A money mule is a bank account, opened by a real customer who is coerced, recruited or paid, that receives criminal proceeds and forwards them onward within hours. Mules sit between the fraud victim and the organiser, and freezing one strands funds that are otherwise irrecoverable. India’s real-time payment rails make this faster and more damaging, and the Reserve Bank of India requires that automated action against a customer account be explainable [1].

Three difficulties compound. Mules are extremely rare, so accuracy is meaningless—predicting that every account is normal scores 99.11% here—and the precision-recall curve is the appropriate summary [2]. Mules are behaviourally camouflaged: the account belongs to a genuine customer with a genuine history, so unsupervised anomaly detection has little purchase, which our ablation confirms sharply. Finally, labels in this domain are the output of an investigation, and that process leaves traces a model will exploit [3].

This last point dominates. While building the detector we found the supplied benchmark cannot support an honest performance estimate, and that the contamination is invisible to standard defences. We therefore present the integrity audit as our primary contribution, with the detector as the vehicle that exposed it.

2. DATASET AND INTEGRITY AUDIT

The dataset comprises 9,082 accounts and 3,924 anonymised features, with a separate dictionary mapping each to a named banking variable. Features follow a regular grammar: a statistic (ratio, deviation, average, extremum) over a payment channel (cash, cheque, UPI, ATM, online transfer, Aadhaar Payment Bridge) in a direction (credit, debit) over a window (7, 14, 31 days). A tail block holds demographics, alert counts by time of day, and investigation metadata.

2.1. Leakage a correlation threshold cannot see

Four fields record an alert’s resolution status and two record how long the investigation took. All six exist only after an analyst closes a case, so none is available when an account must actually be scored. Their correlations with the target span two orders of magnitude: FRAUD_SUSPECTED at 0.969 is caught by any threshold, while FALSE_POSITIVE at 0.055 is caught by none—yet it encodes the same verdict, merely inverted. We therefore classify leakage by what a field means, using the dictionary, and treat correlation only as a backstop.

2.2. A structural confound

Table 1 gives the complete cross-tabulation of class against extract month. Every negative is drawn from the October extract and every positive from September, November and December. No month contains both classes, so the month column alone separates them—and it is not a property of any customer, but a record of which extraction run produced the row.

Table 1. Class composition by monthly extract. No month contains both classes.

Extract month	Normal	Mule
Dec25	0	10
Nov25	0	23
Oct25	9,001	0
Sep25	0	48

Dropping the month column does not help. Because the classes occupy disjoint extracts, any characteristic that varies between extraction runs aligns perfectly with the label while describing nothing about behaviour. Adversarial validation

cannot diagnose this, since a classifier trained to predict the extract is by construction the classifier that predicts the label.

2.3. Falsification tests

We constructed three tests, each supplying a model with information that cannot identify a mule. All should score near the 0.0089 baseline if the data is sound. In test A every value is discarded and the model sees only whether each cell was populated—a property of the extraction job, not the customer. In test B the model receives 250 columns drawn from the 3,174 whose individual correlation is below 0.05, none of which can identify a mule alone. In test C the labels are permuted.

Table 2 reports the outcome. Test C collapsing to the baseline confirms the harness is sound, so the first two results are genuine properties of the data rather than a defect in our code. The conclusion is unavoidable: any figure computed on this dataset, by any team and any method, measures extract provenance in addition to mule behaviour, and the two cannot be separated within this file. Section 4 should be read with that attached.

Table 2. Falsification tests. Information that cannot identify a mule still separates the classes.

Test	Information supplied	AUPRC	AUROC
A	Blank/not-blank only	0.824	0.993
B	250 cols, $ r < 0.05$	0.736	0.973
C	As B, labels shuffled	0.0094	0.528
—	Random guess	0.0089	0.500

3. METHOD

Fig. 1 shows the pipeline. We highlight only what a reader needs to trust the numbers.

Preserving destroyed information. Eight columns are categorical, including occupation, gender and product. Coercing the matrix to numeric—the customary step—converts them to missing values that are then dropped as sparse. This discards real signal: mule rates by occupation range from 0.45% for homemakers to 1.94% for students against a 0.89% base. We encode the account-age bucket ordinally and the rest as indicators. We also revisit imputation: for absolute activity aggregates a missing value means the activity never occurred, so we zero-fill 78,919 values across 1,357 columns rather than inventing activity through median imputation.

Four-layer leakage defence. Layer one removes post-outcome fields semantically. Layer two removes structural artefacts of sample assembly. Layer three, extract hardening, removes any column whose blank rate differs between classes by more than ten percentage points, on the grounds that cell population is decided by the extraction job; this dropped 574 columns. It uses the label, but only ever removes columns, so it can make the reported result worse and never better. Layer four is a separation audit scanning every surviving column for disjoint class ranges, which reports clean after the first three.

Mule-typology features. A mule receives funds and forwards them almost immediately, retains little, operates in bursts, favours digital rails, and belongs to a customer whose profile does not match the volume. We encode each clause as a named family—pass-through ratio, turnover over balance, burst, cash-out share, channel concentration, ticket size, night-alert share, balance volatility, occupation divergence—29 features in total.

Nested validation. The ensemble stacks an isolation forest [4], XGBoost [5] and LightGBM [6] through a logistic meta-learner [7] with isotonic calibration [8]. Every component touching the label—selection, base models, stacking weights, the calibration map, and the decision threshold—is fitted inside the training fold and applied frozen to unseen validation rows. This matters: calibrating on pooled out-of-fold predictions and scoring those same values inflates the result, and selecting the threshold that maximises precision on a curve then reporting that precision is optimistic by construction [9]. We prefer instance reweighting to synthetic oversampling [10], which with 65 training positives across hundreds of dimensions fabricates regions of feature space for which no evidence exists. Because a single split places about 16 mules per fold, we repeat the procedure across shuffles and report mean $\pm$ standard deviation.

4. RESULTS

Table 3 reports the calibrated ensemble. At the precision-first operating point it attains precision 0.980 ± 0.043 at recall 0.719 ± 0.121 , with AUPRC 0.866 ± 0.068 against a 0.0089 baseline; summed across folds, 175 true positives against 4 false positives. A second operating point trades precision to 0.511 ± 0.104 for recall 0.893 ± 0.073 , appropriate when a reviewed alert costs minutes and a missed mule costs a laundering channel.

Table 3. Detection performance and per-model ablation, nested repeated cross-validation (mean $\pm$ std across folds).

Metric	Precision-first	High-recall
Precision	0.980 ± 0.043	0.511 ± 0.104
Recall	0.719 ± 0.121	0.893 ± 0.073
F1	0.823 ± 0.088	0.642 ± 0.087
Per-model AUPRC (threshold-independent)		
Isolation forest		0.007 ± 0.001
LightGBM		0.818 ± 0.069
XGBoost		0.894 ± 0.057
Calibrated ensemble		0.866 ± 0.068

The isolation forest result is the informative one. At AUPRC 0.007 ± 0.001 and AUROC 0.307 it performs worse than random. This is not a defect but a finding, consistent with the criminology: a mule account belongs to a real customer with an ordinary history, and is valuable to the organiser precisely because it does not look anomalous. Unsupervised outlier detection is the wrong tool here, and proposals leading with it deserve suspicion.

Fig. 2 ranks features by mean absolute SHAP contribution [11]. 7 of the top 12 are engineered in this work rather

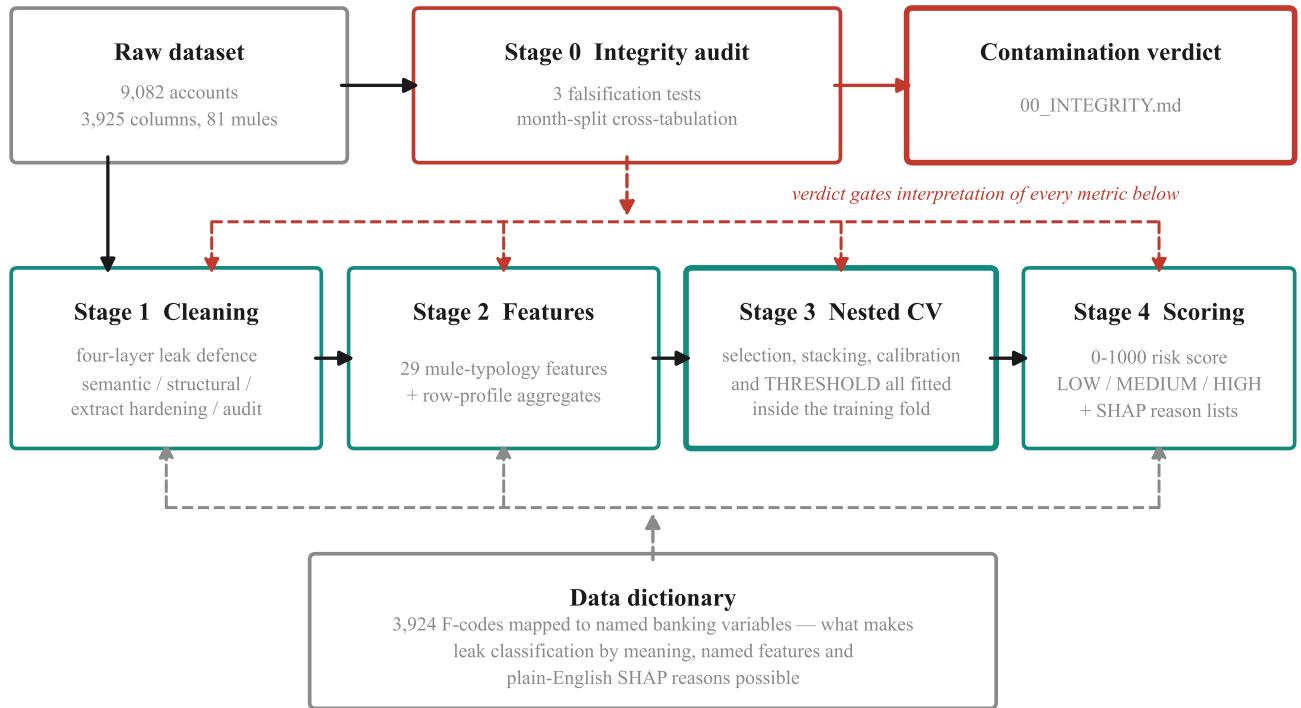


Fig. 1. MuleGuard AI pipeline. Stage 0 runs before any modelling and its verdict qualifies every metric produced downstream (red). Leak defences concentrate in Stage 1. In Stage 3 every component that touches the label — feature selection, stacking weights, the calibration map and the decision threshold — is fitted inside the training fold and applied frozen to validation rows. The data dictionary is drawn as a bus because it is cross-cutting rather than a pipeline step: it supplies the named semantics that Stages 1, 2 and 4 depend on. The graph stage is omitted from the figure; it detects the absence of counterparty identifiers and disables itself.

than supplied columns, indicating the typology encoding contributes materially rather than restating what the raw matrix already exposed. Among supplied columns, deviations of non-cash non-cheque volume from the occupation cohort dominate—the profile-mismatch signal appearing directly.

Operationally, the high-risk band holds 54 accounts of which 54 are confirmed mules (band precision 1.00), capturing 67% of all mules in a queue that is 0.59% of the portfolio. Reviewing well under one percent of accounts surfaces two thirds of the mule population.

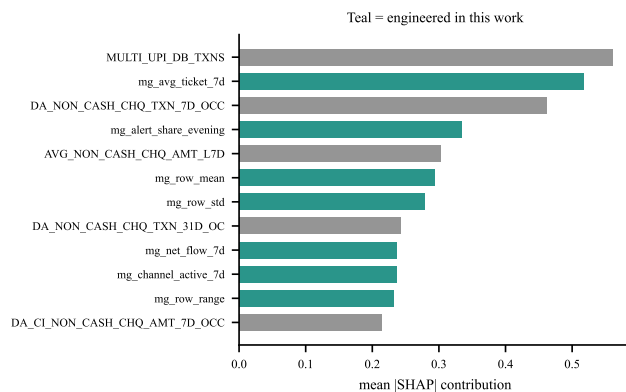


Fig. 2. Top features by mean absolute SHAP value. Features engineered in this work are shown in teal, supplied dictionary variables in grey.

5. DISCUSSION AND CONCLUSION

The results above are strong, and we do not believe they measure mule detection capability. Section 2 establishes that an uninformative view of the same data achieves comparable separation, so a substantial and unquantifiable share reflects the extract a row came from. Our hardening removes the artefact where it is identifiable; it cannot remove what is not. A behavioural feature that also drifts between months is confounded, and no modelling choice separates the two within this file. The remedy is a sampling change, not a modelling one: negatives drawn from the same months as positives would let month be conditioned on. This affects every team working from this release.

Further limitations bear statement. The dataset is an account-level matrix with no counterparty identifiers, so the graph component detects their absence and disables itself rather than fabricating an edge list; network-propagation claims cannot be evaluated here. Hyperparameters remain

at conservative defaults, since tuning against a confounded objective optimises the artefact. And 81 positives is a small sample—the standard deviations reported are wide enough that differences of a few points should not be considered meaningful.

We suggest the falsification protocol of Section 2 deserves wider use. It is inexpensive, requires no knowledge of the modelling approach, and here was the difference between reporting a headline result and understanding what that result measured [12].

6. ACKNOWLEDGMENT

The authors thank the organisers, Bank of India and the Indian Institute of Technology Hyderabad, for the dataset and the accompanying data dictionary. The dictionary proved essential rather than incidental: without documented variable semantics neither the meaning-based leakage taxonomy nor the named feature families could have been constructed. The integrity findings are offered in a collaborative spirit—an observation intended to strengthen the benchmark for all participants and for future releases, not a criticism of those who assembled and shared it.

7. REFERENCES

- [1] Reserve Bank of India, “Master direction on know your customer (KYC) direction, 2016 (updated),” 2016, Reserve Bank of India, Mumbai.
- [2] J. Davis and M. Goadrich, “The relationship between precision-recall and ROC curves,” in Proc. 23rd Int. Conf. Machine Learning, 2006, pp. 233–240.
- [3] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: formulation, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [4] F. T. Liu, K. M. Ting, and Z. Zhou, “Isolation forest,” in Proc. 8th IEEE Int. Conf. Data Mining, 2008, pp. 413–422.
- [5] T. Chen and C. Guestrin, “XGBoost: a scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, “LightGBM: a highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017, vol. 30, pp. 3146–3154.
- [7] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [8] B. Zadrozny and C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates,” in Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2002, pp. 694–699.
- [9] G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [11] S. M. Lundberg and S. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017, vol. 30, pp. 4765–4774.
- [12] Bank of India and Indian Institute of Technology Hyderabad, “PSB cybersecurity, fraud and AI hackathon 2026 problem statement,” 2026, PLACEHOLDER — replace with the organisers’ exact citation.